

astrid-runtime/astrid

Astrid : un runtime capability-secure Rust/WASM techniquement solide, porté par une seule personne — adopter en périphérie, ne pas en dépendre encore

Astrid is a portable, capability-secure operating system for composable software.

1

BUS FACTOR
sur 12 semaines

319

COMMITTS
sur 12 semaines

117

DÉPENDANCES
15 en production

Apache-2.0

LICENCE

<https://github.com/astrid-runtime/astrid> Licence Apache-2.0 Onboard · profil cto

Que fait ce projet et quelle est son architecture ?

Un runtime capability-secure Rust/WASM de 33 crates, 10277 étoiles depuis sa création le 2026-02-15, encore « the hosted runtime, not a bootable OS image » : l'architecture est claire, le produit n'est pas fini.

<code>.cargo/</code>	config Cargo locale
<code>.gemini/</code>	config agent Gemini
<code>.github/</code>	20 workflows GitHub Actions
<code>changes/</code>	notes de changement en attente
<code>container/</code>	définitions de conteneur
<code>crates/</code>	code principal : 33 crates Rust, 32 membres du workspace
<code>docs/</code>	documentation
<code>e2e/</code>	scénarios TOML et fixtures bout en bout
<code>fuzz/</code>	tests de fuzzing
<code>native/</code>	code natif macOS
<code>release/</code>	config de release (nightly.toml)
<code>scripts/</code>	scripts build, CI, release
<code>.dockerignore</code>	exclusions du build Docker
<code>.gitattributes</code>	attributs git
<code>.gitignore</code>	exclusions git
<code>.gitmodules</code>	sous-modules git

... et 12 autres entrées à la racine du dépôt

Astrid se présente comme « a portable, capability-secure operating system for composable software » : aujourd'hui un runtime user-space macOS et Linux, la cible annoncée étant un OS autonome ; la release courante est « the hosted runtime, not a bootable OS image ». Sa thèse : un agent LLM est du code non fiable, la frontière de sécurité doit être dans le runtime, pas dans le prompt.

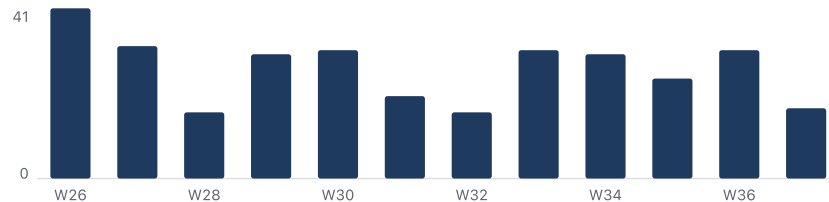
Architecture : un noyau volontairement « dumb » (`astrid-daemon`) qui charge des **capsules** WebAssembly dans Wasmtime, route l'IPC sous ACL de capacités et tient une chaîne d'audit signée ; les frontends (CLI, passerelle HTTP, Discord) sont des **uplinks** sur socket Unix ; l'ABI hôte est le component model avec des paquets WIT `astrid:*` (`fs` , `io` , `kv` , `ipc` , `net` , `http` , `sys` , `process` , `approval` , `identity` , `elicit` , `uplink`). Sécurité décomposée en gates indépendants et fail-closed : sandbox WASM, manifeste allow-list, ACL IPC, jeton ed25519, approbation, sandbox OS (bwrap/seatbelt), audit hash-linké.

Code : un workspace Cargo de 32 membres, 33 crates sous `crates/` , découpés par domaine : noyau (`astrid-kernel` , `astrid-daemon` , `astrid-runtime` , `astrid-events`), sécurité (`astrid-crypto` , `astrid-capabilities` , `astrid-audit` , `astrid-approval` , `astrid-vfs`), stockage (`astrid-storage` + providers FUSE/FSKit/WinFsp), frontends (`astrid-cli` , `astrid-gateway` , `astrid-mcp` , `astrid-uplink`), outillage capsules (`astrid-build` , `astrid-capsule*` , `astrid-emit`). Autour : 20 workflows GitHub Actions, `e2e/` en scénarios TOML, `fuzz/` , `native/macos/` , un `CHANGELOG.md` de 410 Ko.

Chiffres : 10277 étoiles, 137 forks, 216 issues ouvertes, dépôt créé le 2026-02-15, organisation, non archivé. Le SDK des capsules (`astrid-sdk`), le Book, le handbook et les RFC sont dans d'autres dépôts¹² : ils sont hors des données relevées.

Quelle est sa santé technique ?

16 à 41 commits par semaine, une CI qui bloque sur `clippy -D warnings` et `cargo-audit`, v2026.9.0 livrée le 2026-09-09 : le pipeline est sain, mais couverture et taux de succès des runs restent non relevés.



Cadence soutenue et régulière : entre 16 commits et 41 commits par semaine sur les 12 dernières semaines, dernier commit le 2026-09-09. 149 issues fermées sur 30 jours pour 192 ouvertes ; 188 PR fusionnées en 30 jours listées, 24 PR ouvertes.

Releases : v2026.9.0 le 2026-09-09, après une série v0.9.3 → v0.10.4 en juillet et trois canaux `channel-stable` / `nightly` / `dev` . Le saut 0.10 → 2026.9 est un changement de schéma de version (year.month.patch) annoncé dans le README.

CI : `ci.yml` déclenché sur push et pull_request, 14 jobs : `check`, `fmt`, `clippy -D warnings` , `cargo test --workspace --locked` sur ubuntu et macos, MSRV 1.95, `cargo-audit 0.22.2`, builds release Linux gnu/musl 4 cibles, tests natifs Windows (WinFsp épinglé par SHA256), portabilité wasm32, `cargo-semver-checks` et `cargo-public-api` consultatifs sur PR, `harden-runner` sur chaque job. Vingt workflows au total dont `codeql`, `scorecard`, `dependency-review`, `nightly`, `release`. Le README annonce `#![deny(unsafe_code)]` partout sauf `astrid-sys` et `astrid-sdk` , `clippy` pedantic, lint d'overflow arithmétique en erreur, binaires signés Sigstore keyless ; le `Cargo.toml` confirme `unsafe_code = "deny"` et `arithmetic_side_effects = "deny"` .

Tests : `crates/astrid-integration-tests` , 39 fichiers, `cargo test + insta`, commande `cargo test --workspace -- --quiet` . Données non relevées : couverture, durée de CI, taux de succès des runs ; à vérifier sur <https://github.com/astrid-runtime/astrid/actions>.

Bémol : Windows est testé en CI mais exclu des archives 2026.9.0 ; `wit` à la racine est une entrée de 0 octet et `.gitmodules` déclare des sous-modules dont la cible n'est pas relevée.

Qui porte le projet et quel est le bus factor ?

Bus factor 1 : joshuajbouw signe 309 commits sur 12 semaines contre 7 pour le deuxième contributeur, risque high, aucun financement déclaré ; le fork miroir est une assurance, pas une option.



Bus factor : 1. Sur 12 semaines, joshuajbouw signe 309 commits, jvsteiner 7, Copilot 3. Il n'y a que trois contributeurs dans le top 10.

Le Cargo.toml nomme deux auteurs : « Joshua J. Bouw » et « Unicity Labs » ; le dépôt appartient à une organisation et un CODEOWNERS existe. Données non relevées : les autres mainteneurs éventuels et la nature du lien avec Unicity Labs ; à vérifier dans <https://github.com/astrid-runtime/astrid/blob/main/github/CODEOWNERS> et <https://github.com/astrid-runtime/handbook>.

Le risque est classé haut : « Un seul contributeur porte la moitié des commits sur 12 semaines ». Aucun financement déclaré (pas de FUNDING). Parade : ne pas dépendre d'un fork privé, mais sécuriser un accès aux clés de release et aux dépôts satellites, ou négocier un contrat de maintenance avec l'éditeur.

Quelles dépendances et quelle dette ?

117 dépendances workspace avec Wasmtime 48 en pivot, 91 issues `needs-design` et 0 repère IA sur 7 : la dette est dans les contrats non gelés, pas dans les TODO.

Manifeste `Cargo.toml`, 117 entrées dans `[workspace.dependencies]` : 27 crates internes et 90 externes, dont 8 épinglées en version exacte ; `Cargo.lock` versionné (217 Ko) ; transitives non relevées. Niveau de risque déclaré : mid.

Dépendances structurantes : `wasmtime` / `wasmtime-wasi` 48.0.1, la famille `wasm-tools` 0.257.1 (`wasmparser`, `wasm-encoder`, `wit-component`, `wit-parser`, `wat` épinglé `=1.257.1`), `tokio` 1.49, `axum` 0.8, `rustls` 0.23, `ed25519-dalek` 3.0, `blake3` 1.5, `surrealkv` 0.21, `rmcp` ~3.1.4, `sigstore-verify` `=0.11.0`, `landlock` 0.4, et des crates de chunking peu diffusées épinglées (`fastcdc` `=5.0.0` et `=4.0.1` en parallèle, `mincdc` `=0.1.0`, `mothcdc` `=0.7.2`). Exemple d'épingle relevée : `caseless` `=0.2.2` .

Lecture CTO : le couplage à Wasmtime est le plus lourd (montée 47 → 48 faite en août, PR #1786 et #1739) ; deux versions de `fastcdc` cohabitent et trois crates CDC exotiques verrouillées signalent un format de stockage en cours de stabilisation. Deux PR Dependabot attendent : 7 mises à jour d'actions et 14 de crates Rust.

Dette explicite : 0 résultat pour `TODO OR FIXME` mais 13 fichiers pour `TODO` seul, dont `crates/astrid-kernel/src/lib.rs`, `crates/astrid-vfs/src/workspace_cow/overlayfs.rs` et le WIT `astrid-net@1.0.0` ; `FIXME` non compté. Dette implicite : 91 issues `needs-design`, 16 `blocked`, 42 `bug` ; aucun repère IA (`AGENTS.md`, `CLAUDE.md`...) : 0 sur 7.

Quels risques ?

Un seul risque high, le bus factor 1 ; dépendances mid, licence low, CI low ont chacun une parade, et le `SECURITY.md` figé à 0.2.x face à v2026.9.0 est le signal à surveiller.

- **license** Apache-2.0 : licence permissive.

- **bus_factor** Un seul contributeur porte la moitié des commits sur 12 semaines.

- **deps** 117 entrées dans `[workspace.dependencies]` de `Cargo.toml` : 27 crates internes (path) et 90 crates externes, dont 8 épinglées en version exacte (=). `Cargo.lock` versionné (217 Ko). Nombre de d...

- **ci** 20 workflows dans `.github/workflows`. `ci.yml` (push sur main et stream-*, pull_request) enchaîne cargo check, fmt, clippy -D warnings, cargo test --workspace (ubuntu et macos), MSRV 1.95, car...

Quatre risques relevés : licence low, bus factor high, dépendances mid, CI low.

Bus factor — HIGH. Un seul auteur pour 309 des commits, bus factor 1. Parade : conserver un fork miroir et une build reproductible interne, suivre l'organisation plutôt que la personne.

Sécurité déclarée vs prouvée — MID. Périmètre bien cadré dans `SECURITY.md` (échappement de sandbox, forge de jetons, ed25519/blake3, SSRF, audit), réponse sous 48 h et correctif sous 7 jours. Mais le tableau « Supported Versions » s'arrête à 0.2.x alors que la release est 2026.9.0 : politique non tenue à jour. 46 issues portent le label `security` et l'architecture « minimal kernel vs privileged package services » est encore en discussion (#1719, p1, needs-design). Aucun audit externe relevé ; à vérifier dans <https://github.com/astrid-runtime/astrid/security>.

Stabilité des contrats — MID. Trois campagnes en cours (`campaign/os-universal` 32 issues, `campaign/station`), 91 issues `needs-design` , contrats (ABI, IPC, manifeste, SDK) soumis à RFC, semver-checks seulement consultatif. Parade : épingler une release, ne consommer que l'ABI WIT documentée.

Plateformes — MID. Windows hors archives 2026.9.0 ; macOS exige FSKit sur macOS 26+ et une approbation d'extension signée. Parade : cibler Linux GNU/MUSL en premier.

Licence — LOW mais à clarifier. GitHub relève Apache-2.0 ; le README et le `Cargo.toml` disent « MIT OR Apache-2.0 », les deux fichiers sont présents. Double licence permissive, pas de blocage.

Dépendances — MID. Voir section précédente ; cargo-audit et dependency-review en CI limitent l'exposition.

Adopter, contribuer ou forker ?

Adopter en pilote non critique sur Linux, version épinglée v2026.9.0, fork miroir en assurance ; réévaluer quand le bus factor dépasse 1 et que les contrats sortent des RFC.

Roadmap : pas de jalons relevés et thèmes vides ; à vérifier dans <https://github.com/astrid-runtime/astrid/milestones>. Ce qui se lit : 10 PR ouvertes listées dont 7 en draft, et les demandes les plus discutées vont vers l'OS natif et la station de capsules : #1708 « qualify the signed amd64 Linux-backed Astrid host profile » (29 commentaires), #1564 « epic: build the Astrid universal application substrate », #1406 namespaces de capacités dynamiques (blocked), #1664 cérémonie d'enrôlement du premier propriétaire (blocked). Le projet investit dans son futur OS, pas dans la consolidation du runtime hébergé.

Adopter : oui pour un usage en périphérie (sandbox d'outils d'agents, passerelle MCP), sur Linux, version épinglée, avec build interne reproductible ; pas comme socle d'un produit tant que le bus factor reste à 1 et que les contrats sont sous RFC.

Contribuer : possible mais encadré, workflow issue-first, tiers de contributeurs, crates de sécurité réservées au core, refactors externes refusés ; le seul relecteur effectif est aussi l'auteur principal. Utile pour faire entrer des correctifs de plateforme, pas pour peser sur l'architecture.

Forker : la double licence le permet, mais reprendre 33 crates, 117 dépendances et une cadence de 16 à 41 commits/semaine sans l'auteur n'est pas réaliste à court terme.

Recommandation : adopter en pilote non critique dès maintenant, contribuer sur les tests et la plateforme Linux pour gagner le tier Astronaut, garder un fork miroir comme assurance ; réévaluer dans deux releases sur trois signaux : bus factor > 1, `SECURITY.md` à jour, et RFC des contrats gelés.

Décision

- **Adopter en périphérie, ne pas en dépendre.** Astrid entre comme sandbox d'outils d'agents ou passerelle MCP sur Linux GNU/MUSL, jamais comme socle d'un produit tant que le bus factor reste à 1.
- **Conditions d'entrée** : release épinglée v2026.9.0, build interne reproductible à partir du `Cargo.lock` versionné, binaires vérifiés via Sigstore keyless, fork miroir tenu à jour sous double licence MIT OR Apache-2.0.
- **Contribution ciblée** : tests et plateforme Linux, en issue-first, pour atteindre le tier Astronaut ; aucun effort sur les crates de sécurité, réservées aux tiers Core et Maintainer.
- **Réévaluer dans deux releases sur quatre signaux** : bus factor > 1, `SECURITY.md` aligné sur la version courante (0.2.x aujourd'hui, contre v2026.9.0), contrats ABI/IPC/SDK sortis du processus RFC, reflux des 91 issues `needs-design`.
- **Ne pas forker** : reprendre 33 crates, 117 dépendances et 16 à 41 commits par semaine sans l'auteur principal n'est pas tenable.

Sources

1. <https://github.com/astrid-runtime/sdk-rust>
2. <https://github.com/astrid-runtime/rfcs>